

SISTEMAS OPERACIONAIS · PROGRAMAÇÃO CONCORRENTE

Problema Produtor-Consumidor com Semáforos

Implementação multithreaded em C e análise de desempenho

Memória compartilhada limitada · semáforos contadores · exclusão mútua

Estudo de caso: $N \in \{2, 8, 32\}$ · 9 combinações (N_p, N_c) · $M = 10^4$

Medições realizadas em Windows 11, 4 CPUs lógicas

Compilado com GCC (MinGW-w64), POSIX threads

Sumário

1. Sincronização: semáforos e exclusão mútua

2. Estrutura do projeto

3. Implementação em C

4. Camada Python: experimento e gráficos

5. Metodologia experimental

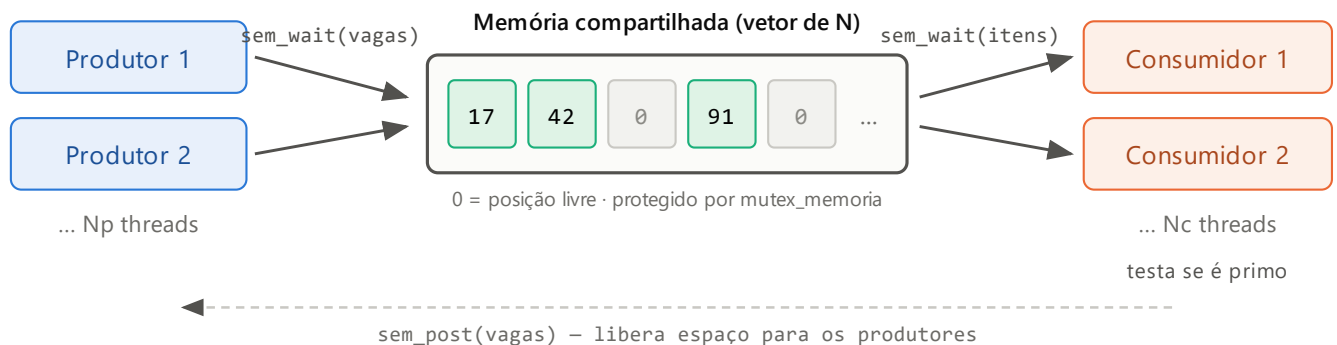
6. Resultados

7. Conclusão

8. Como executar

1. Sincronização: semáforos e exclusão mútua

A memória compartilhada é um vetor de N inteiros, onde 0 denota posição livre. N_p threads produtoras geram inteiros aleatórios entre 1 e 10^7 e os inserem em uma posição livre; N_c threads consumidoras retiram um número, copiam-no para memória local e testam se é primo. O programa termina quando M números tiverem sido processados.



Três primitivas coordenam o acesso ao vetor:

Primitiva	Inicia em	Responde à pergunta
<code>sem_vagas</code> (contador)	N	Existe posição livre? Senão, o produtor dorme.
<code>sem_itens</code> (contador)	0	Existe posição ocupada? Senão, o consumidor dorme.
<code>mutex_memoria</code>	destravado	Exclusão mútua: só uma thread mexe no vetor por vez.

Os dois contadores são complementares: `sem_vagas + sem_itens` é sempre igual a N .

Ordem obrigatória: semáforo contador antes do mutex, nunca o contrário. Invertendo, a thread dormiria segurando o mutex e ninguém mais conseguiria entrar na região crítica para liberá-la — deadlock permanente.

CORRETO



DEADLOCK



2. Estrutura do projeto

```
Produtor-Consumidor/
├─ executar.py           ponto de entrada (menu)
├─ Makefile             atalhos de compilação
├─ README.md
├─ src/
│   └─ produtor_consumidor.c  o programa do enunciado
├─ analise/
│   ├── experimento.py       estudo de caso, CSV e gráfico
│   ├── caminhos.py         caminhos do projeto
│   ├── gerar_pdf.py         gera esta documentação
│   ├── demo_visual.py       animação opcional (Tkinter)
│   └─ simulador_python.py   núcleo em Python, só para a animação
├─ build/
│   └─ produtor_consumidor   (.exe no Windows)
├─ resultados/
│   ├── resultados.csv
│   └─ grafico_desempenho.png
└─ docs/
    ├── Documentacao.pdf
    └─ enunciado.jpeg
```

[src/produtor_consumidor.c](#)

Toda a lógica do problema: threads, semáforos, memória compartilhada, teste de primalidade e cronometragem. Só depende de libc e pthreads.

[analise/experimento.py](#)

Compila o C se necessário, executa o binário para cada uma das 27 configurações (10 vezes cada), lê o tempo pela saída padrão, monta o CSV e o gráfico. Não implementa nada do problema.

[analise/demo_visual.py](#) e [simulador_python.py](#)

Animação opcional em Tkinter do vetor preenchendo/esvaziando, para apoio visual; reimplementação em Python puro, não é a versão avaliada.

3. Implementação em C

3.1 Estruturas de dados

```
typedef struct {
    int *memoria;           // vetor de N inteiros (0 = livre)
    int tamanho;           // N

    sem_t sem_vagas;        // posições livres (inicia em N)
    sem_t sem_itens;        // posições ocupadas (inicia em 0)
    pthread_mutex_t mutex_memoria;

    int total_itens;        // M
    int reservados_producao;
    int reservados_consumo;
    pthread_mutex_t mutex_cota;

    long primos, nao_primos;
    pthread_mutex_t mutex_estatisticas;
    int verboso;
    pthread_mutex_t mutex_saida;
} MemoriaCompartilhada;
```

3.2 Thread produtora

```
while (reservar_item(mc, &mc->reservados_producao)) {
    int valor = gerar_numero(&args->semente);

    sem_wait(&mc->sem_vagas);           // dorme se o vetor estiver cheio

    pthread_mutex_lock(&mc->mutex_memoria);
    int posicao = buscar_posicao_livre(mc);
    mc->memoria[posicao] = valor;
    pthread_mutex_unlock(&mc->mutex_memoria);

    sem_post(&mc->sem_itens);
}
```

3.3 Thread consumidora

```
while (reservar_item(mc, &mc->reservados_consumo)) {
    sem_wait(&mc->sem_itens);           // dorme se o vetor estiver vazio

    pthread_mutex_lock(&mc->mutex_memoria);
    int posicao = buscar_posicao_ocupada(mc);
    int valor = mc->memoria[posicao];    // cópia local
    mc->memoria[posicao] = POSICAO_LIVRE;
    pthread_mutex_unlock(&mc->mutex_memoria);

    sem_post(&mc->sem_vagas);

    int primo = eh_primo(valor);       // fora da região crítica
}
```

3.4 Como o programa termina

Cada thread reserva uma unidade de trabalho de uma cota compartilhada antes de operar sobre o vetor:

```
static int reservar_item(MemoriaCompartilhada *mc, int *contador) {
    int obteve = 0;
    pthread_mutex_lock(&mc->mutex_cota);
    if (*contador < mc->total_itens) {
        (*contador)++;
        obteve = 1;
    }
    pthread_mutex_unlock(&mc->mutex_cota);
    return obteve;
}
```

Como são reservados exatamente M itens de produção e M de consumo, nenhuma thread fica esperando por algo que nunca chega. Verificado automaticamente em cada uma das 270 execuções do estudo.

3.5 Decisões de projeto

- **RNG por thread** (xorshift64* em vez de `rand()`): evita que o estado global do `rand()` vire um ponto de serialização artificial entre as threads.
- **Primalidade fora da região crítica**: o valor é copiado para a pilha da thread antes do teste, para não prender as demais threads durante o cálculo.

4. Camada Python: experimento e gráficos

O C implementa o problema; o Python mede e grafica. O contrato entre os dois é a última linha impressa pelo C:

```
RESULT <tempo_em_segundos> <primos> <nao_primos> <consumidos>
```

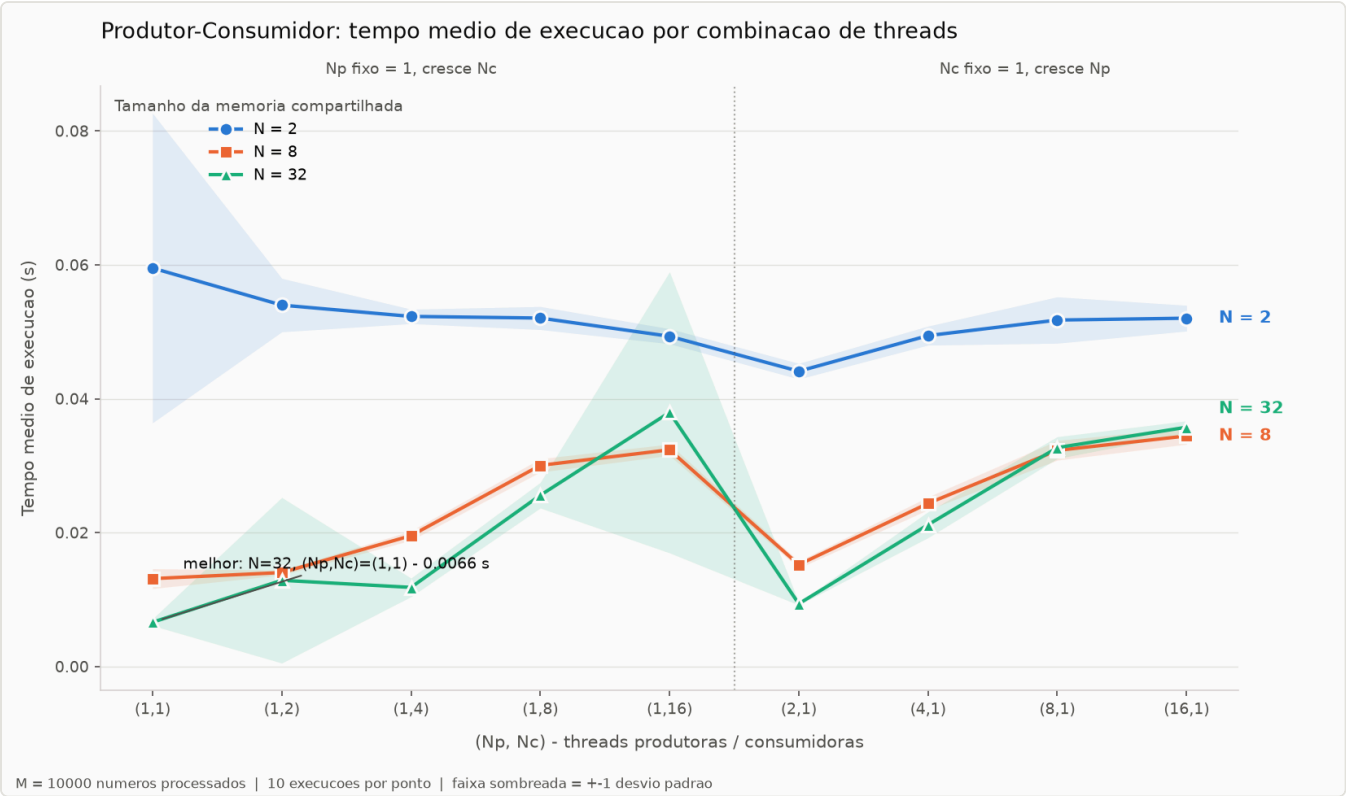
O tempo é cronometrado dentro do C (`clock_gettime(CLOCK_MONOTONIC)`), em volta da criação e do `join` das threads — assim não inclui o custo do sistema operacional criar o processo.

5. Metodologia experimental

Parâmetro	Valor
Números processados (M)	10^4
Tamanho da memória compartilhada (N)	2, 8 e 32
Combinações (N_p , N_c)	(1,1) (1,2) (1,4) (1,8) (1,16) (2,1) (4,1) (8,1) (16,1)
Repetições por combinação	10
Total de execuções	$3 \times 9 \times 10 = 270$
Faixa dos números sorteados	1 a 10^7
Compilação	<code>gcc -O2 -Wall -Wextra -static -pthread</code>
Máquina	Windows 11, 4 CPUs lógicas

As nove combinações formam dois blocos: nas cinco primeiras $N_p=1$ e N_c cresce; nas quatro últimas $N_c=1$ e N_p cresce — por isso o gráfico traz uma linha divisória entre os dois blocos. A impressão no terminal fica desligada durante as medições (a E/S mascararia o tempo real); ela só é usada no modo `--verbose`.

6. Resultados



Tempo médio de execução por combinação de threads, uma curva para cada valor de N.

6.1 Tabela completa

(Np, Nc)	N = 2	N = 8	N = 32
(1,1)	0.0595	0.0131	0.0066
(1,2)	0.0540	0.0140	0.0129
(1,4)	0.0523	0.0195	0.0118
(1,8)	0.0520	0.0300	0.0256
(1,16)	0.0493	0.0324	0.0380
(2,1)	0.0441	0.0152	0.0094
(4,1)	0.0494	0.0244	0.0212
(8,1)	0.0517	0.0323	0.0327
(16,1)	0.0520	0.0344	0.0357

Tempo médio de execução em segundos. Em destaque, a combinação de melhor desempenho.

6.2 Efeito de aumentar produtores x consumidores

Quantidade de threads	Aumentando N_c (com $N_p=1$)	Aumentando N_p (com $N_c=1$)
1	0.0264	0.0264
2	0.0270	0.0229
4	0.0279	0.0317
8	0.0359	0.0389
16	0.0399	0.0407

Tempo médio em segundos, calculado sobre os três valores de N. Em destaque, o melhor de cada coluna.

7. Conclusão

Melhor combinação: $N=32$, $(N_p, N_c)=(1,1)$ — 0.0066 s (9.0× mais rápida que a pior, $N=2$, $(N_p, N_c)=(1,1)$, 0.0595 s).

- **N domina o resultado:** de $N=2$ para $N=32$ o tempo médio cai de 0.0516 s para 0.0215 s (2.4×). Com N pequeno o vetor vive cheio ou vazio e quase toda operação encontra o semáforo em zero — cada bloqueio custa uma chamada ao escalonador, mais caro que o próprio teste de primalidade.
- **Mais threads pioraram o desempenho**, dos dois lados: toda inserção e toda retirada passam pelo mesmo mutex, uma região crítica serial por construção (Lei de Amdahl). A máquina de teste tem 4 CPUs lógicas.
- **Não há M que reverta o quadro.** Com M cem vezes maior (10^6 , $N=32$) o tempo continua crescendo com mais threads:

(N_p, N_c)	Tempo com $M = 10^6$
(1, 1)	1,00 s
(1, 2)	1,05 s
(1, 4)	1,90 s
(1, 8)	2,91 s

Na melhor combinação, cada número custa cerca de 0.66 microssegundos do início ao fim — a maior parte é sincronização, não cálculo (~94% dos sorteios entre 1 e 10^7 são compostos, descartados nas primeiras divisões). Como o trabalho útil por item é menor que o custo de sincronizar o acesso a ele, nenhuma quantidade de threads compensa; paralelismo só valeria a pena com um custo por item bem maior (números maiores, ou lotes por entrada na região crítica).

8. Como executar

Requisitos: GCC com POSIX threads (pacote `gcc` no Linux; MinGW-w64 variante *posix* no Windows) e Python 3 com `pandas` e `matplotlib`.

```
# menu com todas as opções
python executar.py

# compilar manualmente (Linux)
gcc -O2 -Wall -Wextra -o build/produtor_consumidor \
    src/produtor_consumidor.c -pthread

# compilar manualmente (Windows; -static evita depender de uma DLL)
gcc -O2 -Wall -Wextra -static -o build/produtor_consumidor.exe \
    src/produtor_consumidor.c -pthread

# demonstração: imprime cada número e se é primo
build/produtor_consumidor 8 2 2 20 --verbose

# estudo de caso completo do enunciado
python analise/experimento.py

# variações
python analise/experimento.py --rapido
python analise/experimento.py --M 1000000 --repeticoes 3
```

```
produtor_consumidor <N> <Np> <Nc> <M> [--verbose]

N   tamanho da memória compartilhada (vetor)
Np  número de threads produtoras
Nc  número de threads consumidoras
M   quantidade de números a processar
--verbose imprime cada número consumido e se é primo
```